PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Улучшение модели — данные, признаки, проверка

> Привет! На прошлом занятии вы собрали полный мини-пайплайн машинного обучения. Но первая модель редко бывает лучшей. Сегодня вы будете действовать как команда, которая получила неудовлетворительный результат и должна улучшить его без сложной математики: найти проблемы в данных, убрать бесполезные признаки, заново проверить модель и доказать улучшение числами. К концу занятия вы сможете объяснить не только «стало лучше», но и почему это произошло.
> Важно не пытаться исправить всё одновременно. Если изменить данные, признаки, алгоритм и способ проверки за один шаг, будет трудно определить причину нового результата. Поэтому улучшение модели будет проходить как последовательность небольших проверяемых изменений.

Содержание

- 1. Чему вы научитесь
- 2. Среда выполнения и правила честного сравнения
- 3. Краткий теоретический минимум
- 4. Подготовка рабочего шаблона
- 5. Чек-лист перед сдачей
- 6. Итог урока

1. Чему вы научитесь

- отличать проблему данных от проблемы алгоритма;
- находить пропуски, дубликаты, невозможные значения и шумовые столбцы;
- выбирать признаки, связанные с задачей;
- сравнивать модели на одной и той же тестовой выборке;

- использовать accuracy, macro F1 и матрицу ошибок;
- повторно проверять качество с помощью нескольких разбиений;
- формулировать вывод об улучшении и его ограничениях.

Эти умения образуют единый цикл работы: сначала проводится аудит данных, затем формируется подходящий набор признаков, после чего модель обучается и оценивается по заранее выбранным правилам. Если результат недостаточно устойчив, цикл повторяется с новым обоснованным изменением.

2. Среда выполнения и правила честного сравнения

При работе в ноутбуке результат одной ячейки может зависеть от всех выполненных ранее ячеек. Нарушение порядка запуска иногда приводит к ситуации, когда на экране остаются старые переменные или метрики, не соответствующие текущему коду. Поэтому воспроизводимость эксперимента начинается с одинакового порядка действий.

- Выполняйте ячейки сверху вниз и не удаляйте RANDOM_STATE = 42. Фиксированное значение помогает повторить случайное разбиение и получить сопоставимые результаты.
- Не изменяйте тестовую выборку: исходная и улучшенная модели должны проходить один и тот же «экзамен».
- Меняйте по одному или по небольшой группе понятных факторов и записывайте результат. Например, сначала можно исключить ID и шумовые столбцы, а затем отдельно обработать ошибки в значениях.
- Не считайте улучшением случайный рост одной метрики на одном запуске. Рост accuracy следует сопоставить с macro F1, матрицей ошибок и результатами повторной проверки.

Обратите внимание

Значение RANDOM_STATE не делает модель лучше. Оно фиксирует случайные операции, чтобы эксперимент можно было повторить. Если изменить это значение, состав обучающей и тестовой выборок может измениться, а значит, прямое сравнение с предыдущим запуском станет некорректным.

3. Краткий теоретический минимум

3.1. Три способа улучшить результат

Низкая оценка модели ещё не указывает на конкретную причину. Ошибка может находиться в исходной таблице, в наборе признаков или в способе проверки. Разделение работы на три направления помогает не делать случайных изменений и связывать каждый результат с определённым действием.

Направление	Что проверяем	Примеры действий
Данные	Можно ли доверять строкам и значениям?	Удалить полные дубликаты; обработать пропуски; найти невозможные значения
Признаки	Есть ли в таблице полезный сигнал?	Убрать ID и случайный шум; сравнить разные наборы признаков
Проверка	Улучшение устойчиво или случайно?	Использовать тот же тест; выполнить кросс-валидацию; сравнить несколько метрик

Проблемы разных направлений могут встречаться одновременно. Например, в одной таблице могут присутствовать и пропуски в полезных признаках, и технический столбец с номером строки. Пропуски требуют обработки значений, а технический столбец обычно требует исключения из набора признаков.
После каждого изменения полезно задавать два вопроса: какую конкретную проблему оно устраняет и по каким показателям будет проверяться результат? Если на первый вопрос нет точного ответа, изменение пока недостаточно обосновано.

3.2. Почему «добавим более сложную модель» — не первый шаг

Если в таблице есть пропуски, дубликаты, ошибочные значения и случайные столбцы, более сложный алгоритм может лишь точнее запомнить беспорядок. Сначала разумно улучшить то, что подаётся модели, и только потом менять сам алгоритм.
Например, столбец **sample_id** обозначает номер объекта, а не его свойства. Числа в таком столбце могут выглядеть как обычный числовой признак, хотя их порядок не описывает класс объекта. Аналогично столбцы **noise_1**–**noise_8** создаются случайно и не содержат целевого сигнала по смыслу учебного примера.
Частая ошибка — считать, что большее количество столбцов автоматически даёт модели больше полезной информации. На практике важно не число признаков само по себе, а их связь

с решаемой задачей и корректность значений.

3.3. Что значит честное сравнение

Исходная и улучшенная модели должны проверяться на одних и тех же тестовых объектах. Это похоже на две команды, которые решают одинаковый вариант контрольной. Если варианты разные, нельзя уверенно сказать, кто справился лучше.
Одинаковым должен оставаться не только список тестовых объектов, но и правильные ответы **y_test**. Кроме того, метрики следует вычислять одинаковым способом. Нельзя, например, сравнивать accuracy одной модели с macro F1 другой и делать вывод о превосходстве по двум несопоставимым числам.
Тестовую выборку не используют для выбора признаков, заполнения пропусков или настройки шагов обработки. Эти действия должны определяться по обучающим данным. Иначе сведения о тесте попадут в процесс обучения, и оценка перестанет отражать работу модели на действительно независимых объектах.

Мини-проверка
Можно ли честно сравнить две модели, если первая проверена на 38 объектах, а вторая — на других 38 объектах? Нет: различие метрик может быть связано не с моделями, а с неодинаковой сложностью двух наборов объектов.

3.4. Метрики занятия

Одна метрика показывает только одну сторону результата. Поэтому в практикуме используются числовые оценки общего качества и матрица, которая позволяет рассмотреть ошибки по отдельным классам.

Метрика	Простой смысл	На что обратить внимание
Accuracy	Доля правильных прогнозов	Удобна при примерно равных классах
Macro F1	Среднее качество по каждому классу	Не позволяет крупному классу «скрыть» слабый результат по другому классу
Матрица ошибок	Таблица правильных и ошибочных прогнозов	Показывает, какие классы модель путает

Если модель правильно классифицировала 30 объектов из 38, её accuracy равна 30 / 38, то есть примерно 0,789. Однако это число не показывает, к каким классам относились восемь ошибочных прогнозов.

Macro F1 сначала учитывает результат для каждого класса отдельно, а затем усредняет полученные значения с одинаковым весом. Поэтому слабый результат по одному из классов нельзя полностью скрыть хорошими прогнозами для остальных классов.
В матрице ошибок правильные прогнозы располагаются на главной диагонали, а значения вне диагонали соответствуют ошибкам. При чтении матрицы важно заранее проверить принятый порядок классов и расположение истинных и предсказанных меток.

> **Важно**
> **Более высокая accuracy не гарантирует улучшения по каждому классу. Если accuracy выросла, а macro F1 снизилась, необходимо изучить матрицу ошибок и выяснить, для какого класса качество стало хуже.**

4. Подготовка рабочего шаблона

Создайте новый ноутбук и выполните два блока кода. Они формируют одинаковые данные у всех обучающихся.
Первый блок подключает инструменты для работы с таблицами, разбиения данных, построения Pipeline, заполнения пропусков, масштабирования, обучения модели и вычисления метрик. Переменная **rng** создаёт отдельный генератор случайных чисел с фиксированным состоянием.

Блок 1. Импорт библиотек

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix

RANDOM_STATE = 42
rng = np.random.RandomState(RANDOM_STATE)
```

После выполнения блока в памяти появляются названия библиотек, классов и функций, но модель ещё не обучается. Если на этом этапе возникает ошибка импорта, следующие ячейки также не смогут выполниться.

Блок 2. Подготовка независимого теста и «испорченной»

обучающей таблицы

```
iris = load_iris(as_frame=True)
X = iris.data.copy()
y = iris.target.copy()

# Сначала создаём единый независимый тест.
X_train_clean, X_test, y_train_clean, y_test = train_test_split(
    X, y,
    test_size=0.25,
    random_state=RANDOM_STATE,
    stratify=y
)

# Учебная копия обучающих данных с типичными проблемами.
X_train_bad = X_train_clean.reset_index(drop=True).copy()
y_train_bad = y_train_clean.reset_index(drop=True).copy()
X_test_base = X_test.reset_index(drop=True).copy()
y_test = y_test.reset_index(drop=True)

X_train_bad["sample_id"] = np.arange(1001, 1001 + len(X_train_bad))
X_test_base["sample_id"] = np.arange(9001, 9001 + len(X_test_base))

for i in range(1, 9):
    X_train_bad[f"noise_{i}"] = rng.normal(0, 100 * i, len(X_train_bad))
    X_test_base[f"noise_{i}"] = rng.normal(0, 100 * i, len(X_test_base))

for column in X.columns:
    rows = rng.choice(X_train_bad.index, size=18, replace=False)
    X_train_bad.loc[rows, column] = np.nan

rows = rng.choice(X_train_bad.index, size=10, replace=False)
X_train_bad.loc[rows, "sepal length (cm)"] *= 8

train_bad = X_train_bad.copy()
train_bad["target"] = y_train_bad
train_bad = pd.concat(
    [train_bad, train_bad.sample(20, random_state=1)],
    ignore_index=True
)
```

Разбиение выполняется до добавления учебных проблем. Параметр **test_size=0.25** выделяет четверть исходных объектов для тестирования, а **stratify=y** помогает сохранить соотношение классов при разбиении. Тестовые ответы хранятся отдельно в **y_test**.
Метод **reset_index(drop=True)** создаёт последовательные индексы строк и не добавляет старый индекс как новый столбец. Вызов **copy()** нужен для получения отдельных копий таблиц, которые можно изменять в учебном эксперименте.
Затем в данные добавляются технический идентификатор и восемь шумовых столбцов. В исходные признаки обучающей части вносятся пропуски, а часть значений длины чашелистика умножается на 8, что создаёт значения, требующие проверки. В конце 20 строк повторно добавляются в таблицу вместе с целевой меткой, поэтому появляются полные дубликаты объектов.
Переменные **X_train_clean** и **y_train_clean** сохраняют исходную обучающую часть.
Переменная **train_bad** содержит учебную таблицу с признаками, целевой меткой и добавленными проблемами. Не следует путать её с независимой тестовой частью.

Важно
Проблемы добавляются только в учебных целях. В реальном проекте нельзя искусственно ухудшать данные или подменять результаты.
Нельзя исправлять учебную таблицу с помощью сведений из y_test. Тестовые метки предназначены только для итоговой оценки прогнозов.

Перед дальнейшей работой проверьте, что оба блока выполнены без ошибок и в памяти существуют переменные **train_bad**, **X_test_base** и **y_test**. Повторный запуск второго блока восстановит учебные данные в соответствии с зафиксированным состоянием генераторов случайных чисел.

5. Чек-лист перед сдачей

Проверяйте работу после перезапуска ноутбука и последовательного выполнения всех ячеек. Это помогает обнаружить зависимости от старых переменных и случайно пропущенных действий.

- Все ячейки выполняются сверху вниз без ошибок.
- Исходная и улучшенная модели проверены на одном тесте.
- Заполнена таблица проблем данных.
- Удалены дубликаты и невозможные значения; пропуски обработаны внутри Pipeline.
- ID и шумовые столбцы не используются в улучшенной модели.
- Сравнены минимум два набора признаков.

- Указаны accuracy, macro F1 и матрица ошибок.
- Проведена повторная проверка или заполнен её готовый шаблон.
- Итоговый вывод содержит числа, причины улучшения и ограничения.

Отдельно убедитесь, что значения метрик подписаны и понятно, к какому варианту модели они относятся. Названия вроде «до обработки» и «после обработки» должны сопровождаться точным описанием выполненных изменений.

6. Итог урока

> Сегодня вы улучшили модель не за счёт «магии» и не за счёт случайной замены алгоритма. Вы проверили данные, отобрали признаки и повторили оценку. Главный результат практикума: качество модели зависит не только от алгоритма, но и от того, какие данные ему передали и насколько честно проверили результат.
> Корректный итог эксперимента отвечает на три вопроса: что было изменено, как изменились метрики и насколько устойчиво наблюдаемое различие. Такой вывод позволяет отличить обоснованное улучшение от случайного удачного запуска.